Assignment 3: Hybrid MPI + OpenMP Numerical Integration

Name: Eddie Ogedno

ID: 3736377

Date: April 2026

Github: [edd-ie/Asn3-CS4575]

1. Introduction

The objective of this assignment was to extend the MPI-based adaptive numerical integration from Assignment 2 into a **Hybrid MPI + OpenMP** solution. By combining distributed memory parallelism (MPI) with shared memory parallelism (OpenMP Tasks), the system aims to evaluate the trade-offs between dynamic global load balancing and static hybrid distribution for irregular numerical workloads.

2. Methodology

2.1 Hardware and Software Environment

- **Machine:** Intel(R) Core(TM) i5-8250U @ 1.60GHz | x86_64 (4 Cores / 8 Threads)
- **Environment:** Ubuntu 24.04 via WSL
- **Compilation:** mpicc -fopenmp -O3 integration.c -lm
- **MPI Version:** Open MPI 5.0.9

2.2 Implementation Design

- **Mode 1 (MPI Dynamic):** Reused the Master-Worker architecture. Rank 0 manages a global task stack. Workers (Ranks 1 to P-1) request work and send back sub-intervals when recursion is required. This ensures high load balancing but incurs significant communication overhead.
- **Mode 2 (Hybrid Static):** The interval [0, 1] is statically partitioned into P equal segments (where P is the number of MPI processes). Each process independently computes its segment using **OpenMP Tasks** to handle local recursion.
- **OpenMP Strategy:** Within each MPI worker, #pragma omp parallel and #pragma omp single are used to initiate the first recursive call. Subsequent splits are handled via #pragma omp task and synchronized with #pragma omp taskwait.

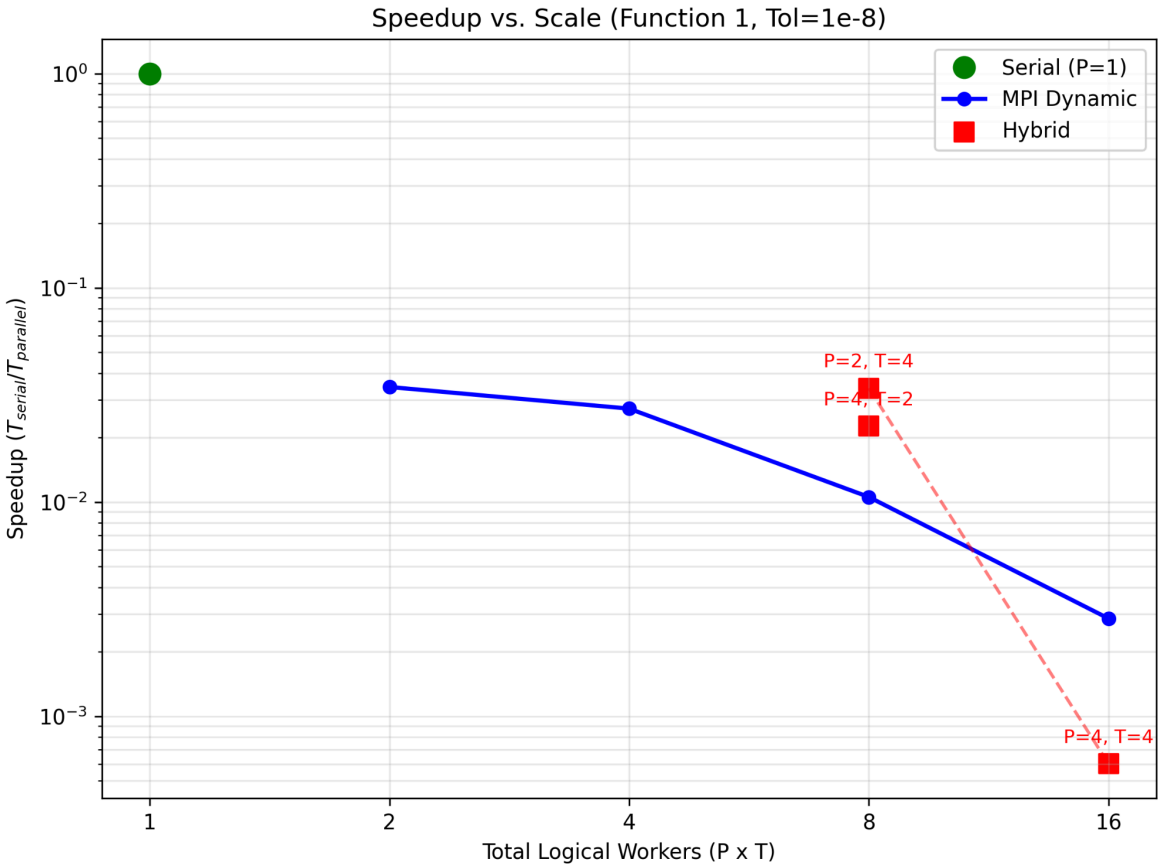
3. Performance Results

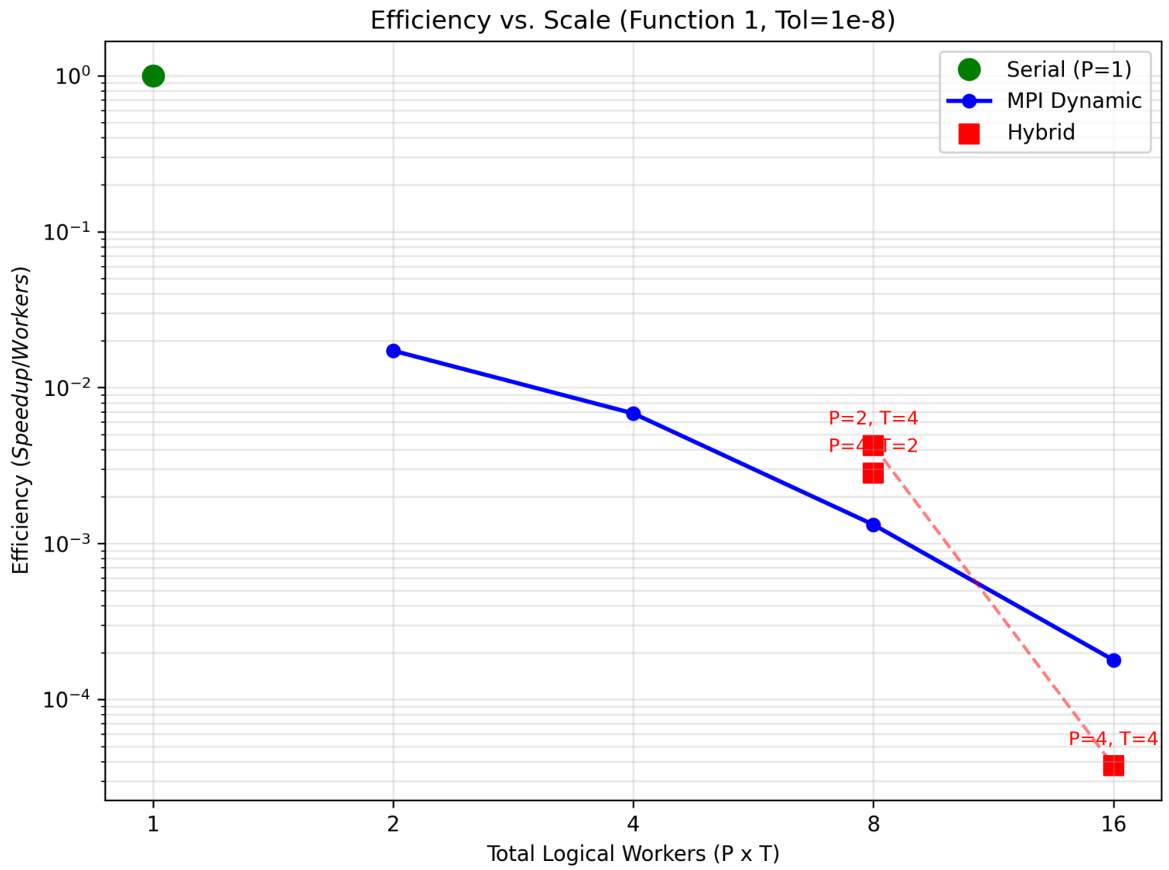
The following table summarizes the performance for **Function 1** at a tolerance of 1e-8.

Speedup $S = T_{serial} / T_{parallel}$

Efficiency $E = S / (P \times T)$.

Mode	Processes (P)	Threads (T)	Workers	Ave Time (s)	Speedup (S)	Efficiency (E)
Serial	1	1	1	0.000005	1	1
MPI	2	1	2	0.000145	0.034	0.017
MPI	4	1	4	0.000183	0.027	0.006
MPI	8	1	8	0.000473	0.01	0.001
MPI	16	1	16	0.001746	0.002	0.0001
Hybrid	2	4	8	0.000147	0.034	0.004
Hybrid	4	2	8	0.000221	0.022	0.002
Hybrid	4	4	16	0.008321	0.0006	0.00003


Speedup vs. Scale (Function 1, Tol=1e-8)
Serial (P=1)
MPI Dynamic
Hybrid
P=2, T=4
P=4, T=2
P=4, T=4
Speedup ($T_{serial}/T_{parallel}$)
10^{0}
10^{-1}
10^{-2}
10^{-3}
1
2
4
8
16
Total Logical Workers (P x T)



4. Discussion

4.1 Why does MPI dynamic perform well for irregular workloads?

MPI Dynamic scheduling excels in irregular workloads because it prevents **process starvation**. In functions with "spikes", some intervals require much deeper recursion than others. A dynamic master-worker queue ensures that as soon as a process finishes an "easy" interval, it can immediately assist with the workload generated by "hard" intervals, maintaining high CPU utilization across the cluster.

4.2 Why might Hybrid perform worse?

In the observed results for Function 1, Hybrid performance degraded significantly as process and thread counts increased (notably the P=4, T=4 run). For computationally light functions, the overhead of managing the OpenMP task queue and the cost of thread synchronization exceeds the time required to simply calculate the integral serially.

4.3 Does OpenMP compensate for the lack of MPI load balancing?

Only partially. Because the MPI processes are statically assigned ranges (Mode 2), OpenMP cannot help if one MPI process is assigned a complex region of the function while another process is assigned a simple region. The idle MPI process cannot "steal" tasks from the busy one.

4.4 Impact of Thread vs. Process Count

The data shows that (P=2, T=4) outperformed (P=4, T=2) and (P=4, T=4). This suggests that on this specific hardware, increasing the number of MPI processes (distributed memory) introduces more context-switching or communication penalties than increasing threads (shared memory) up to a certain point. However, oversubscribing threads (P x T = 16 on a machine with fewer cores) results in severe performance collapse.

5. Authorship Trail

commit 0bb5e9a8bf60614b1fcd8602df6ece94435b4243 (HEAD -> main, origin/main)

- Documentation and project clean up

commit 95e1d3ae01a96b9caba7a661bbd952a987915b08 (HEAD -> main)

- Updated script with the --oversubscribe to allow exceed the machine's physical core count

commit fed39553bf830247bda883fb2b580648ebd550b6

- fix MPI_Recv blocking error

commit b2e27858c156f8b44afd735cd6e5bad80845c46f

- Update to the makefile

commit 44839beb16999652288bf848b7abb02506bfd0de

- mode 1 rank 0 refinement

commit e63d3a42ca4752e1cfb6d5df7b5bfb6d5bf26692

- Finalized Hybrid Mode (Mode 2) and implemented the 10-run average timing loop.

commit 7010f0d2e01df93eb687e9418ae5e51cf3f7108d

- Mode 2 framework

commit aaf054d0dcae8a2f066525dcfc676189ca72a181

- Hybrid Worker Function

commit 24a5aa17b34ecc4f2e30dc3e0133d85b3807c71a (origin/main)

- MPI Mode restructure

commit 76a548f6187bd1d09367d3ef7e96a1b19ea66ae0

- Assignement 2 structure

commit b380472457c45ca8b998baf39b0842739dc9a0a7

- First Commit